

Papers

- **SWE-chat (Baumann et al., 2026)**: Stanford's empirical study of 6,000 real-world coding agent sessions, revealing a highly bimodal split between "vibe coding" (agents writing 99% of code in 41% of sessions) and human-led efforts (23% of sessions), while showing that only 44% of AI-generated code survives to commits.
- **CodeTracer (Li et al., 2026)**: Introduces a framework that reconstructs heterogeneous agent run artifacts into hierarchical "trace trees" (exploration vs. state-changing nodes), allowing developers to perform failure onset localization to pinpoint exactly where an agent went off-track.
- **AgentStepper (SOLA-ST, 2026)**: The first interactive debugger for LLM coding agents, mapping classical software debugging (breakpoints, stepwise execution, and live prompt/argument editing) to agent trajectories and repository-level code modifications.
- **View-oriented Conversation Compiler (VCC) (Zhang et al., 2026)**: A compiler pipeline that translates verbose, raw agent JSONL logs into structured, optimized views (Full lossless, User Interface, and Compacted), cutting evaluator token costs by 50% to 67% and boosting task success.
- **Nautilus-Compass (2026)**: Analyzes "prompt drift" in long agent sessions (where models forget instructions or fabricate APIs) and deploys a real-time defense that measures cosine similarity between active prompts and behavioral anchors to trigger calibration alerts.

Competitors & Tools

- **LangSmith (Type 1)**: LangChain's enterprise observability platform designed to screen agent traces at scale, grouping trajectories, monitoring tool loops or argument errors, and running LLM-as-a-judge trajectory evaluations.
- **Langfuse (Type 1)**: An open-source, OpenTelemetry-native tracing and evaluation platform that tracks multi-turn conversations and visually renders multi-step reasoning paths as "agent graphs".
- **Arize Phoenix (Type 1)**: An open-source, OTel-compatible evaluation tool that isolates nested trace spans (agent, LLM, tool) and runs programmatic evaluations to classify whether tool-calling sequences are correct and efficient.
- **agentsview (Type 2)**: A local-first, zero-account session browser that auto-discovers and indexes logs from over 20 coding platforms into local SQLite, DuckDB, or PostgreSQL databases, providing instant full-text search and cost tracking.
- **ai-blame (Type 2)**: A terminal and desktop utility that extracts audit trails from local transcripts to generate chronological action timelines and Git-blame-style, line-by-line file attribution.
- **AiderDesk (Type 2)**: An open-source graphical workbench for the Aider CLI featuring isolated Git worktree sandboxing, smart context vector search, chat history curation, and human tool-approval gates.

- **CCHV (Type 2):** A cross-platform desktop history viewer that supports multi-agent indexing (Claude Code, Cursor, Cline, etc.), full-text cross-session search, and estimated cost dashboards.

Datasets & Specifications

- **SWE-chat Dataset:** A living, growing database of 6,000+ real-world agent sessions capturing full conversation transcripts, tool calls, private thinking, and precise human-versus-AI line-level code attribution.
- **Agent Trace Specification (v0.1.0 RFC):** A vendor-neutral metadata standard (`application/vnd.agent-trace.record+json`) backed by Cursor, Cognition, and Vercel to bind AI code attribution and conversation URIs to specific line ranges and VCS commit SHAs.
- **CodeTraceBench:** A diagnostic benchmark aggregating thousands of annotated trajectories generated by four widely used agent frameworks, serving to evaluate step-level failure localization.

Technical Documentation

- **Claude Code Session Format:** Technical details on how Claude Code saves transcripts locally as append-only, newline-delimited JSONL files under home directories, structuring conversations as a Directed Acyclic Graph (DAG) using parent UUID links.
- **pi-vcc (Pi Conversation Compactor):** An open-source, algorithmic conversation compactor for the Pi platform that runs locally in 30 to 470ms without LLM calls, merging files, preferences, and rolling transcript windows to achieve a 35% to 99% token footprint reduction with zero API costs.